

Introduction

This short PDF is all about Laravel Eloquent. We love it, we use it, but there are a lot of hidden gems!

For example:

- Did you know about Eloquent's strict mode?
- Or invisible database columns?
- Or did you know that you can write an attribute accessor and mutator in one method using the Attribute cast?
- ...And don't forget about the Prunable trait!

On the following pages, you can read about my **39 favorite Eloquent recipes**.

I also released other high-rated materials, you can [find them here](#).

Basic Eloquent Recipes

Disable Lazy Loading

Dealing with N+1 queries is probably one of the most common sources of performance issues. Since Laravel 8, you can entirely turn off lazy loading. This means if a relationship is not eager loaded, and you try to access it, Laravel will throw an exception in your face!

```
class AppServiceProvider extends ServiceProvider
{
    public function boot()
    {
        Model::preventLazyLoading(App::isLocal());
    }
}
```

You can pass a condition, and if it's false, Laravel won't throw exceptions. It's a "best practice" to only disable it in development environment, so your server won't explode.

Strict Mode (new in Laravel 9)

Laravel 9 introduced strict mode which is even stricter than `preventLazyLoading` because it will:

- Disable lazy loading.
- Throw an error if you try to set an attribute that is not present in the `fillable` array. This is a great feature to have while developing a project.
- Disable using model attributes that hasn't been fetched from the DB yet. This is pretty useful to avoid N+1 queries.
- Disable accessing non-existing attributes.

Here's an example for the third one:

```

class Post extends Model
{
    public function getLatestCommentAttribute()
    {
        return $this->comments()->latest()->first();
    }
}

foreach (Post::all() as $post) {
    $latestComment = $post->latest_comment;
}

```

Accessing the `latest_comment` property will run an additional query so you'll end up with an N+1 query problem. Of course you can prevent it with the `shouldBeStrict` method:

```

class AppServiceProvider extends ServiceProvider
{
    public function boot()
    {
        Model::shouldBeStrict(App::isLocal());
    }
}

```

If you want to avoid N+1 queries in projects where you cannot turn these features on check out [Laracheck](#).

Routing and SoftDeletes (new in Laravel 9)

When you have a model that uses the `SoftDeletes` trait you can query the soft-deleted model like this:

```
Post::where('author_id', $authorId)
    ->withTrashed();
```

You probably already knew that. But did you know you can use the function with an entire route?

```
Route::get('posts/{post}', [PostController::class, 'show'])
    ->withTrashed();
```

In this case route model binding will query not just active but also soft-deleted records.

Since Laravel 9.35 we can use this with resource routes as well:

```
Route::resource('posts', PostController::class)
    ->withTrashed();
```

Now `withTrashed` will apply to every method that has a *post* parameter. In fact, you can even tell Laravel in which method you want to use it:

```
Route::resource('posts', PostController::class)
    ->withTrashed(['show']);
```

Benchmarking (new in Laravel 9)

This feature is not strictly related to Eloquent but it is the important use-case for it. Since Laravel 9.32 we have a new helper class called *Benchmark* which can be used such as this:

```
class OrderController
{
    public function index()
    {
        return Benchmark::measure(fn () => Order::all());
    }
}
```

It'll run the callback function and then prints out the time required to run it. You can also measure multiple tasks:

```
return Benchmark::measure([
    fn () => Order::all(),
    fn () => Order::where('status', 'pending')->get(),
]);
```

Invisible Database Columns

The invisible column is a new concept in MySQL 8. What it does: when you run a `select *` query it won't retrieve any invisible column. If you need an invisible column's value you have to specify it explicitly in the `select` statement.

And now, Laravel supports these columns:

```
Schema::table('users', function (Blueprint $table) {
    $table->string('password')->invisible();
});

$user = User::first();
$user->secret === null;
```

This query will return `null` despite the fact that it's a `select *` query. It's because `password` is invisible.

If you need the `password` you need to run this query:

```
$user = User::select('password')->first();  
$user->password === null;
```

It's a very useful concept for sensitive information such as:

- Password
- Tokens
- API keys
- Payment-related information

saveQuietly

If you ever need to save a model but you don't want to trigger any model events, you can use this method:

```
$user = User::first();
$user->name = 'Guest User';
$user->saveQuietly();
```

Default Attribute Values

In Laravel you can define default values for columns in two places:

- Migrations
- Models

First, let's see the migration:

```
Schema::create('orders', function (Blueprint $table) {
    $table->bigIncrements('id');
    $table->string('status', 20)
        ->nullable(false)
        ->default(App\Enums\OrderStatuses::DRAFT);
});
```

This is a well-known feature. The `status` column will have a default `draft` value.

But what about this?

```
$order = new Order();
$order->status === null;
```

In this case, the status will be `null`, because it's not persisted yet. And sometimes it causes annoying `null` value bugs. But fortunately, you can specify default attribute values in the Model as well:

```
class Order extends Model
{
    protected $attributes = [
        'status' => App\Enums\OrderStatuses::DRAFT,
    ];
}
```

And now the `status` will be `draft` for a new Order:

```
$order = new Order();
$order->status === 'draft';
```

You can use these two approaches together and you'll never have a `null` value bug again.

Attribute Cast

Before Laravel 8.x we wrote attribute accessors and mutators like these:

```
class User extends Model
{
    public function getNameAttribute(string $value): string
    {
        return Str::upper($value);
    }

    public function setNameAttribute(string $value): string
    {
        $this->attributes['name'] = Str::lower($value);
    }
}
```

It's not bad at all, but as Taylor says in the pull request:

This aspect of the framework has always felt a bit "dated" to me. To be honest, I think it's one of the least elegant parts of the framework that currently exists. First, it requires two methods. Second, the framework does not **typically** prefix methods that retrieve or set data on an object with `get` and `set`

So he recreated this feature this way:


```

use Illuminate\Database\Eloquent\Casts\Attribute;

class User extends Model
{
    protected function name(): Attribute
    {
        return new Attribute(
            get: fn (string $value) => Str::upper($value),
            set: fn (string $value) => Str::lower($value),
        );
    }
}

```

The main differences:

- You have to write only one method
- It returns an `Attribute` instead of a scalar value
- The `Attribute` itself takes a getter and a setter function

In this example, I used PHP 8 named arguments (the `get` and `set` before the functions).

find

Everyone knows about the `find` method, but did you know that it accepts an array of IDs?

So instead of this:

```
$users = User::whereIn('id', $ids)→get();
```

You can use this:

```
$users = User::find($ids);
```

Get Dirty

In Eloquent you can check if a model is "dirty" or not. Dirty means it has some changes that are not persisted yet:

```
$user = User::first();  
$user->name = 'Guest User';  
  
$user->isDirty() === true;  
$user->getDirty === ['name' => 'Guest User'];
```

The `isDirty` simply returns a bool while the `getDirty` returns every dirty attribute.

push

Sometimes you need to save a model and its relationship as well. In this case, you can use the `push` method:

```
$employee = Employee::first();  
$employee->name = 'New Name';  
$employee->address->city = 'New York';  
  
$employee->push();
```

In this case the, `save` would only save the `name` column in the `employees` table but not the city column in the `addresses` table. The `push` method will save both.

Boot Eloquent Traits

We all write traits that are being used by Eloquent models. If you need to initialize something in your trait when an event happened in the model, you can boot your trait.

For example, if you have models with slug, you don't want to rewrite the slug creation logic in every model. Instead, you can define a trait, and use the `creating` event in the boot method:

```
trait HasSlug
{
    public static function bootHasSlug()
    {
        static::creating(function (Model $model) {
            $model->slug = Str::slug($model->title);
        });
    }
}
```

So you need to define a `bootTraitName` method, and Eloquent will automatically call this when it's booting a model.

updateOrCreate

Creating and updating a model often use the same logic. Fortunately Eloquent provides a very convenient method called `updateOrCreate`:

```
$flight = Flight::updateOrCreate(
    ['id' => $id],
    ['price' => 99, 'discounted' => 1],
);
```

It takes two arrays:

- The first one is used to determine if the model exists or not. In this example, I use the `id`.
- The second one is the attributes that you want to insert or update.

And the way it works:

- If a `Flight` is found based on the given `id` it will be updated with the second array.
- If there's no `Flight` with the given `id` it will be inserted with the second array.

I want to show you a real-world example of how I handle creating and updating models.

The Controller:

```
public function store(UpsertDepartmentRequest $request): JsonResponse
{
    return DepartmentResource::make($this->upsert($request, new Department()))
        ->response()
        ->setStatusCode(Response::HTTP_CREATED);
}

public function update(
    UpsertDepartmentRequest $request,
    Department $department
): HttpResponse {
    $this->upsert($request, $department);
    return response()->noContent();
}

private function upsert(
    UpsertDepartmentRequest $request,
    Department $department
): Department {
    $departmentData = new DepartmentData( ... $request->validated());
    return $this->upsertDepartment->execute($department, $departmentData);
}
```

As you can see I often extract a method called `upsert`. This method accepts a `Department`. In the `store` method I use an empty `Department` instance because in this case, I don't have a real one. But in the `update` I pass the currently updated instance.

The `$this->upsertDepartment` refers to an Action:

```
class UpsertDepartmentAction
{
    public function execute(
        Department $department,
        DepartmentData $departmentData
    ): Department {
        return Department::updateOrCreate(
            ['id' => $department->id],
            $departmentData->toArray(),
        );
    }
}
```

It takes a `Department` which is the model (an empty one, or the updated one), and a DTO (a simple object that holds data). In the first array I use the `$department->id` which is:

- `null` if it's a new model.
- A valid ID if it's an updated model.

And the second argument is the DTO as an array, so the attributes of the `Department`.

upsert

Just for confusion Laravel uses the word `upsert` for multiple update or create operations. This is how it looks:

```
Flight::upsert([
    ['departure' => 'Oakland', 'destination' => 'San Diego', 'price' => 99],
    ['departure' => 'Chicago', 'destination' => 'New York', 'price' => 150]
], ['departure', 'destination'], ['price']);
```

It's a little bit more complicated:

- First array: the values to insert or update
- Second: unique identifier columns used in the select statement
- Third: columns that you want to update if the record exists

So this example will:

- Insert or update a flight from Oakland to San Diego with the price of 99
- Insert or update a flight from Chicago to New York with the price of 150

when

We often need to append a `where` clause to a query based on some conditional, for example, a Request parameter. Instead of `if` statements you can use the `when` method:

```
User::query()
    ->when($request->searchTerm, function ($query) {
        $query->where('username', 'LIKE', "%$request->searchTerm%");
    })
    ->get();
```

It will only run the callback if the first parameter returns true.

The same snippet without `when`:

```
$query = User::query();
if ($request->searchTerm) {
    $query->where('username', 'LIKE', "%$request->searchTerm%");
}

return $query->get();
```

appends

If you have an attribute accessor and you often need it when the model is converted into JSON you can use the `$appends` property:

```
class Product extends Model
{
    protected $appends = ['current_price'];

    public function getCurrentPriceAttribute(): float
    {
        return $this->prices
            ->where('from', '<=' now())
            ->where('to', '>=' now())
            ->first()
            ->price;
    }
}
```

Now the `current_price` column will be appended to the `Product` model every time it gets converted into JSON. It's useful when you're working with Blade templates. With APIs, I would stick to Resources.

Relationship Recipes

whereRelation

Imagine you're working on a financial application like a portfolio tracker, and you have the following models:

- Stock: each company has a stock with a ticker and a current price.
- Holding: each holding has some stocks in their portfolios. It has columns like `invested_capital`, `market_value`

A Holding belongs to a Stock and a Stock has many Holdings. You can write a simple join to get every Apple holdings, for example:

```
$apple = Holding::select('holdings.*')
    →leftJoin('stocks', 'stocks.id', 'holdings.stock_id')
    →where('stocks.ticker', 'AAPL')
    →get();
```

Or you can use the `whereRelation` helper:

```
$apple = Holding::whereRelation('stock', 'ticker', 'AAPL')→get();
```

It reads like this: give every Holdings where the Stock relation's ticker column is equal to AAPL (this is the ticker symbol of Apple). Under the hood, it will run an `EXISTS` query. So if you actually need data from the stocks table it's not a good solution.

whereBelongsTo

Consider this snippet:

```
public function index(User $user)
{
    $sum = Order::where('user_id', $user->id)
        ->sum('total_amount');
}
```

It looks good. How about this?

```
public function index(User $user)
{
    $sum = Order::whereBelongsTo($user)
        ->sum('total_amount');
}
```

With Eloquent you almost speak in English.

oldestOfMany

There's a special relationship called `oldestOfMany`. You can use it if you constantly need the oldest model from a `hasMany` relationship.

In this example, we have an `Employee` and a `Paycheck` model. An employee has many paychecks, so this is the basic relationship:

```
class Employee extends Models
{
    public function paychecks()
    {
        return $this->hasMany(Paycheck::class);
    }
}
```

If you want to get the oldest paycheck you don't have to write a custom query every time, you can create a relationship for it:

```
class Employee extends Models
{
    public function oldestPaycheck()
    {
        return $this->hasOne(Paycheck::class)
            ->oldestOfMany();
    }
}
```

In this case, you have to use the `hasOne` method because it will return only one paycheck, the oldest one. The oldest paycheck is the one with the smallest auto-increment ID. So it won't work if you are using UUIDs as foreign keys.

latestOfMany

Similarly to `oldestOfMany` we can use the `newestOfMany` as well:

```
class Employee extends Models
{
    public function latestPaycheck()
    {
        return $this->hasOne(Paycheck::class)
            ->latestOfMany();
    }
}
```

The latest paycheck is the one with the largest auto-increment ID.

ofMany

You can also use the `ofMany` relationship with some custom logic, for example:

```
class User extends Authenticable
{
    public function mostPopularPost()
    {
        return $this->hasOne(Post::class)->ofMany('like_count', 'max');
    }
}
```

This will return the post with the highest `like_count`.

So instead of writing a custom query every time, you can use these relationships:

- `oldestOfMany`
- `latestOfMany`
- `ofMany`

hasManyThrough

Often we have relationships like `$parent->child->child`. For example, a department has employees, and each employee has paychecks. This is a simple `hasMany` relationship:

```
class Department extends Model
{
    public function employees(): HasMany
    {
        return $this->hasMany(Employee::class);
    }
}

class Employee extends Model
{
    public function paychecks(): HasMany
    {
        return $this->hasMany(Paycheck::class);
    }
}
```

If we need all the paychecks within a department we can write:

```
$department->employees->paychecks;
```

Instead of this, you can define a `paychecks` relationship on the `Department` model with the `hasManyThrough`:

```
class Department extends Model
{
    public function employees(): HasMany
    {
        return $this->hasMany(Employee::class);
    }

    public function paychecks(): HasManyThrough
    {
        return $this->hasManyThrough(Paycheck::class, Employee::class);
    }
}
```

The second argument is the "through" model. And now you simply write:

```
$department->paychecks;
```

hasManyDeep

Okay, this is clickbait, because there is no `hasManyDeep` relationship in Laravel but there is an excellent package called [eloquent-has-many-deep](#).

Consider the following relationships:

`Country` -> has many -> `User` -> has many -> `Post` -> has many -> `Comment`

If you try to get every comment from a country with Eloquent it will cost you around 1 billion database queries. But with this package you can query every comment in one query using Eloquent:

```
class Country extends Model
{
    use \Staudenmeir\EloquentHasManyDeep\HasRelationships;

    public function comments()
    {
        return $this->hasManyDeep(Comment::class, [User::class, Post::class]);
    }
}
```

So it can handle additional levels of relationships. Under the hood it uses subqueries.

withDefault

Let's say you have a Post model in your application that has an Author relationship which is a User model. Users can be deleted but often we need to keep their data. This means Author is a nullable relationship, and probably cause some code like this:

```
$authorName = $post->author
    ? $post->author->name
    : 'Guest Author';
```

We obviously want to avoid code like this. Fortunately PHP provides us the nullable operator, so we can write this:

```
$authorName = $post->author?->name || 'Guest Author';
```

The hardcoded `Guest Author` seems like an anti-pattern, and you have to write this snippet every time you need the author's name. In this situation you can use the `withDefault`:

```
class Post extends Model
{
    public function author(): BelongsTo
    {
        return $this->belongsTo(User::class)
            ->withDefault([
                'name' => 'Guest Author'
            ]);
    }
}
```

Two things happening here:

- If the `author_id` in the Post is `null`, the `author` relationship will not return `null` but a new `User` model.
- The new `User` model's name is `Guest User`.

So now, you don't need to check `null` values anymore:

```
// Either a real name or 'Guest Author'
$authorName = $post->author->name;
```

Order by Related Model's Average

In this example, we have a `Book` and a `Rating` model. A book has many ratings. Let's say we need to order the books by the average rating. We can use the `withAvg` method:

```
public function bestBooks()
{
    Book::query()
        ->withAvg('ratings as average_rating', 'rating')
        ->orderByDesc('average_rating');
}
```

There's a lot of ratings here so let's clear this up:

- `ratings as average_rating`
 - We want the averages from the ratings table
 - We want an alias called `average_rating`
- `rating`
 - This is the column in the `ratings` table that want to average

Eager Loading Specific Columns

`select *` queries can be slow and memory-consuming. If you want to eager a relationship but you don't need every column, you can specify which ones you want to load:

```
Product::with('category:id,name')->get();
```

In this case, Eloquent will run a `select id, name from categories` query.

saveMany

With the `saveMany` function you can save multiple related models in one function call.

Consider the following relationships: a Product has many Prices.

When you update a Product you may want to delete all the prices and save the new ones. In this scenario you can use the `saveMany`:

```
$productPrices = collect($prices)
    =>map(fn (array $priceData) => new ProductPrice([
        'from_date' => $priceData['fromDate'],
        'to_date' => $priceData['toDate'],
        'price' => $priceData['price'],
    ]));

$product->prices()->delete();
$product->prices()->saveMany($productPrices);
```

It will create all the prices in one query, and you don't have to deal with loops:

```
foreach ($productPrices as $price) {
    $product->prices()->save($price);
}
```

createMany

Similarly to `saveMany` you can also use the `createMany` if you don't have models, but arrays instead:

```
$prices = [
    ['from' => '2022-01-10', 'to' => '2022-02-28', 'price' => 9.99],
    ['from' => '2022-03-01', 'to' => null, 'price' => 14.99],
];

$product->prices()->createMany($prices);
```


Factory and Migration Recipes

foreignId & constrained

This is the default way you write a foreign key in a migration:

```
$table->foreignId('category_id')  
    ->references('id')  
    ->on('categories');
```

It's not bad, but here's a more cool approach:

```
$table->foreignId('category_id')->constrained();
```

`constrained` will call: `references('id')->on('categories')`

Even better, you can do this:

```
$table->foreignIdFor(Category::class)->constrained();
```

nullOnDelete

If you have a nullable relationship just use the `nullOnDelete` helper:

```
$table->foreignId('category_id')  
    ->nullable()  
    ->constrained()  
    ->nullOnDelete();
```

afterCreating

There's an `afterCreating` method on the `Factory` class that you can use to do something after a Model has been created.

When I have Users with profile pictures, I always use this feature:

```
class UserFactory extends Factory
{
    public function definition()
    {
        return [
            'username' => $faker->username,
        ];
    }

    public function configure()
    {
        return $this->afterCreating(function (User $user) {
            $faker = FakerFactory::create();
            $dir = storage_path('images');
            $path = $faker->image($dir, 640, 640, null, false);

            $user->profile_picture_path = $dir . DIRECTORY_SEPARATOR . $path;
            $user->save();
        });
    }
}
```

Now any time you create a new User with a factory, a fake image will be created and the `profile_picture_path` will be set:

```
$user = User::factory()->create();
$user->profile_picture_path === null;
```

Factory For

Let's say you want to create a Product with a Category:

```
$category = Category::factory()>create();  
$product = Product::factory([  
    'category_id' => $category->id,  

```

Instead of creating the category and passing it as an attribute, you can do this:

```
$product = Product::factory()  
    >for(Category::factory())  
    >create();
```

You can use this method for belongs to relationships.

Factory Has

With the `has` method you can do the inverse of the relationship. So you can use this for has many relationships:

```
Category::factory()  
    >has(Product::factory()>count(10));
```

You can also specify the relationship's name if it's different from the table name:

```
Product::factory()  
    >has(ProductPrice::factory(), 'prices');
```

In this example, the `prices` is the name of the relationship on the `Product` model.

Factory States

When working with factories in tests (or seeders) we often need a specific 'state' in a given model. Let's say we have a `Product` model and it has an `active` column.

You can do this:

```
class ProductFactory extends Factory
{
    public function definition()
    {
        return [
            'name' => $this->faker->words(3, true),
            'description' => $this->faker->paragraph(3),
            'category_id' => fn () => Category::factory()->create()->id,
            'active' => !!rand(0, 1),
        ];
    }
}
```

With this setup you have to specify the `active` flag every time you want to set it:

```
$product = Product::factory([
    'active' => false,
])->create();
```

But factories provide a way to define 'states' for your model. A state can be something like an inactive product:

```
class ProductFactory extends Factory
{
    public function definition()
    {
        return [
            'name' => $this->faker->words(3, true),
            'description' => $this->faker->paragraph(3),
            'category_id' => fn () => Category::factory()->create()->id,
            'active' => !!rand(0, 1),
        ];
    }

    public function inactive(): ProductFactory
    {
        return $this->state(fn (array $attributes) => [
            'active' => false,
        ]);
    }
}
```

And now you can create an inactive product like this:

```
$product = Product::factory()->state('inactive')->create();
```

It's very useful when your state affects 3-4 columns.

Other Eloquent Related Recipes

Log Every Database Query

We often want to see every database query that was executed in a request during development. There are multiple solutions, but here's the most simple one:

```
class AppServiceProvider extends ServiceProvider
{
    public function boot()
    {
        if (App::environment('local')) {
            DB::listen(fn ($query) => logger(
                Str::replaceArray('?', $query->bindings, $query->sql)
            ));
        }
    }
}
```

This snippet will log every database query to your default log file if your environment is set to `local`. If you need a more robust solution check out:

- Telescope
- Clockwork
- Laravel Debugbar

whenLoaded

API resource is a really great concept, but it has a bottleneck: it's very easy to create N+1 query problems.

Let's say we have an `Employee` and a `Department` model. An employee belongs to a department.

Imagine the following Controller method for the `GET /employees` API:

```
public function index()
{
    $employees = Employee::all();
    return EmployeeResource::collection($employees);
}
```

Pretty standard. Now let's see the `EmployeeResource`:

```
class EmployeeResource extends JsonResource
{
    public function toArray(Request $request): array
    {
        return [
            'id' => $this->uuid,
            'fullName' => $this->full_name,
            'department' => $this->department,
        ];
    }
}
```

If you have 500 employees you just made **501 queries** with this setup. Let's see what's happening:

```
$employees = Employee::all();
```

This will run the following query:

```
select *
from employees
```

This line:

```
return EmployeeResource::collection($employees);
```

Will convert every `Employee` model to an array, so practically it's a `foreach`. And this line in the resource:

```
'department' => $this->department,
```

Will run the following query:

```
select *
from department
where id = ?
```

For every employee. This is why you have 501 queries and this is why it's called N+1 query problem.

Don't worry! As always, Laravel can help us. Instead of using the `$this->department` directly in the resource, we can use the `whenLoaded` helper:

```
class EmployeeResource extends JsonResource
{
    public function toArray(Request $request): array
    {
        return [
            'id' => $this->uuid,
            'fullName' => $this->full_name,
            'department' => $this->whenLoaded('department'),
        ];
    }
}
```

It will check if the department relationship is already loaded. If it's not, it won't run an additional query. In this case, the `department` index will be `null`.

Great, but now we don't return the departments, so it's empty in the user list on the frontend. To solve this problem, we have to eager load the department relationships in the controller:


```
public function index()
{
    $employees = Employee::with('department')->get();
    return EmployeeResource::collection($employees);
}
```

In this case, Laravel will run one `select` and it will query all the employees with the department (using a subquery).

Pagination Links With Query String

Laravel has built-in pagination that is very easy to use. In this example, there's a `Thread` and a `Category` model. A thread has a category. This controller method returns every thread in a category:

```
class ThreadController extends Controller
{
    public function index(Category $category)
    {
        return $category
            ->threads()
            ->paginate();
    }
}
```

This works fine, and will return pagination URLs like this: `/api/category/abcd-1234/threads?page=1`. However, there is a problem: if you have any query string in your URL it will be lost.

So if your URL looks like this: `/api/category/abcd-1234/threads?filter[title]=laravel`

You still have pagination links like the one above, so without the `filter[title]=laravel` query string.

To solve this problem we can write this:

```
class ThreadController extends Controller
{
    public function index(Category $category)
    {
        return $category
            ->threads()
            ->paginate()
            ->withQueryString();
    }
}
```

It will append any query string to the URL, so your pagination link will be: `/api/category/abcd-1234/threads?page=1&filter[title]=laravel`

Prunable Trait

If you have a logic in your application that deletes some old or unused rows from the database, you can refactor it using the Prunable trait provided by Laravel:

```
class Order extends Model
{
    use Prunable;

    public function prunable(): Builder
    {
        return Order::query()
            ->whereStatus('abandoned')
            ->where('created_at', '<=', now()->subMonth());
    }
}
```

So in your model, you can define when a record is 'prunable'.

And you don't need to write your own command, you only need to schedule the one provided by Laravel:

```
// In your Console\Kernel.php
$schedule->command('model:prune')->daily();
```

Custom Query Builders

With Eloquent you can define your own query builders for your models. By using this you can move your scopes and queries from your models.

First, you have to define a new class:

```
use Illuminate\Database\Eloquent\Builder;

class HoldingBuilder extends Builder
{
    public function whereTicker(string $ticker): self
    {
        return $this->whereRelation('stock', 'ticker', $ticker);
    }
}
```

`whereTicker` is a scope that you can use on your models. The main difference is that we need to return an instance of `self`.

Second, we need to tell Eloquent that the `Holding` model has its own Builder class:

```
class Holding extends Model
{
    use HasFactory;
    use SoftDeletes;

    protected $guarded = [];
    protected $with = ['stock'];

    public function newEloquentBuilder($query): HoldingBuilder
    {
        return new HoldingBuilder($query);
    }
}
```

And now we can use it as a standard Eloquent query:

```
$holding = Holding::whereTicker($ticker)→first();  
$holding→portfolio_id = $id;  
$holding→save();
```

Of course, you can chain other Eloquent methods:

```
Holding::query()  
    →whereTicker('AAPL')  
    →whereBelongsTo($user)  
    →where('created_at', '≤', now())  
    →get();
```

Thank You

Thank you very much for reading this short PDF! I hope you have learned a few cool techniques.

By the way, I'm Martin Joo, a software engineer since 2012. I'm passionate about Domain-Driven Design, Test-Driven Development, Laravel, and beautiful code in general.

I've published my biggest project so far, [Domain-Driven Design with Laravel](#).